

Project 4: Write-Ahead Logging and Crash Recovery

CMPE 321, Introduction to Database Systems, Spring 2026

Deadline: 09 June 2026, 17:00

1 Goal

In this project you will extend the modular DBMS engine from Project 3 with a **Recovery Manager** implementing Write-Ahead Logging (WAL) and a simplified version of the ARIES crash recovery algorithm. The engine must survive arbitrary process kills and recover to a consistent state in which committed transactions persist and uncommitted ones leave no trace.

Project 3 already prepared the ground: the `DiskSpaceManager` exposes a `log.write` stub, the `WriteResult` object carries before-image and after-image data, and `log.csv` is persistent. In this project you will turn those hooks into a fully functional Recovery Manager.

We will test your engine by running `archive.py` with different scenario files that end in a forced crash. By restarting the engine on the same data directory, we expect recovery to reconstruct a consistent state automatically.

2 File Structure

Extend your Project 3 submission with a new module and supporting test material:

```
your_submission/
|-- archive.py           # Updated: builds the engine, runs
    recovery, processes input
|
|-- disk_space_manager/  # From Project 3 (minor modifications)
|-- buffer_manager/     # From Project 3 (modified for WAL #1)
|-- file_index_manager/  # From Project 3 (calls recovery on every
    page change)
|-- query_processor/     # From Project 3 (wraps each op in a
    transaction)
|
|-- recovery_manager/    # NEW
|   |-- __init__.py      # exports RecoveryManager
|   |-- (your files)
|
|-- config.json
|-- README.md
|-- report.pdf
|-- ai_usage.md
|-- log.csv
|-- individual_contribution.pdf
```

3 Architectural Note: Recovery Is a Cross-Cutting Concern

Project 3 had a strictly layered architecture: each layer called only the layer directly below it. The Recovery Manager **does not fit this pattern**, and this is intentional. Recovery is a *cross-cutting concern*: it must interact with multiple layers in different ways.

- **Top-down calls (normal layering):**

- `QueryProcessor` → `RecoveryManager`: transaction control (`begin`, `commit`).
- `FileIndexManager` → `RecoveryManager`: log an update before applying it to a page.
- `RecoveryManager` → `DiskSpaceManager`: write the log file directly (bypassing the buffer pool).

- **Cross-cutting calls:**

- `BufferManager` → `RecoveryManager`: enforce WAL #1 by flushing log records before flushing the corresponding dirty page.
- `RecoveryManager` → `BufferManager`: during recovery, fetch and modify data pages to apply Redo and Undo.

The log file is owned by the Recovery Manager and *must not* pass through the buffer pool. Cached log pages defeat the purpose of WAL: a log record only becomes a recovery guarantee once it is on disk.

4 The Recovery Manager

Exported class: `RecoveryManager`

Constructor: `RecoveryManager(config: dict, disk: DiskSpaceManager)`

The Recovery Manager owns all logging and recovery logic. As with Project 3, we treat it as a black box. What matters is the contract it offers and the WAL invariants it enforces.

4.1 Required Responsibilities

- Generate monotonically increasing Log Sequence Numbers (LSNs). LSNs must survive restarts: do not reset to zero on startup.
- Maintain an in-memory **Transaction Table** with one entry per active transaction (XID, status, lastLSN).
- Maintain an in-memory **Dirty Page Table** with one entry per dirty page (pageID, recLSN).
- Maintain a **log buffer** and a **flushedLSN** counter.
- Write log records of the following types: `update`, `commit`, `end`, `begin_checkpoint`, `end_checkpoint`.
- Provide `flush_log_up_to(lsn)` that the BufferManager calls before flushing any dirty page (WAL #1).
- Run periodic fuzzy checkpoints (interval set by config).
- Run the three-phase recovery on startup.

4.2 The Two WAL Invariants

These are non-negotiable. Your code must enforce both.

1. **WAL #1 (Atomicity):** A dirty data page with `pageLSN = L` must not be written to disk unless `flushedLSN ≥ L`. The BufferManager must call `recovery.flush_log_up_to(page.pageLSN)` before any dirty page write.
2. **WAL #2 (Durability):** `commit(xid)` must not return until every log record up to and including the commit record has been written to disk *and* fsync'd.

4.3 Log Record Format

You may use any serialization format (text or binary), but each record must contain at least the following fields:

Field	Meaning
<code>lsn</code>	Log Sequence Number (unique, monotonically increasing)
<code>prev_lsn</code>	LSN of the previous log record of the same transaction
<code>xid</code>	Transaction identifier
<code>type</code>	One of <code>update</code> , <code>commit</code> , <code>end</code> , <code>begin_chkpt</code> , <code>end_chkpt</code>
<code>page_id</code>	(update only) which page was modified
<code>offset</code>	(update only) byte offset within the page
<code>before</code>	(update only) before-image of the changed bytes
<code>after</code>	(update only) after-image of the changed bytes

4.4 Page-Level Requirement: `pageLSN`

Every data page must carry a `pageLSN` in its header. When the `FileIndexManager` modifies a page, it must:

1. Call `recovery.log_update(...)` with the change.
2. Receive the new LSN as the return value.
3. Write that LSN into the page's `pageLSN` field.
4. Apply the change to the page bytes.

The `pageLSN` is what makes Redo idempotent: a record whose LSN is less than or equal to the page's `pageLSN` has already been applied.

5 The Three-Phase Recovery Algorithm

On startup, before processing any input, the Recovery Manager must run the three phases of ARIES recovery.

5.1 Phase 1 — Analysis

Scan the log forward starting from the LSN recorded in the master record (the `begin_checkpoint` LSN of the most recent checkpoint).

- Load the Transaction Table and Dirty Page Table snapshots from the `end_checkpoint` record.
- For each log record encountered, update the tables as if the operation were happening live.
- At the end: the Transaction Table contains the **losers** (transactions active and uncommitted at the moment of the crash).

5.2 Phase 2 — Redo (“Repeat History”)

Scan the log forward starting from the *smallest* `recLSN` in the Dirty Page Table. Reapply every update record *unless* one of the following holds:

- The affected page is not in the Dirty Page Table.
- The record's LSN is less than the page's `recLSN`.
- The page's current `pageLSN` on disk is greater than or equal to the record's LSN.

After Redo, the database reflects the state at the moment of the crash *including* the changes made by losers. Undo will clean those up.

5.3 Phase 3 — Undo

Build the set of `lastLSN` values from every loser transaction. Repeatedly pick the largest LSN in the set, read the corresponding update record, apply its before-image to the affected page, and replace the LSN in the set with the record's `prev_lsn`. When a transaction's chain is exhausted (`prev_lsn` is `None`), write an `end` record for that transaction. Continue until the set is empty.

Finally, for any transaction in the Transaction Table whose status is `committed` but which has no `end` record yet (it committed just before the crash but didn't get to clean up), write an `end` record for it as well.

One-shot recovery. This project's recovery is one-shot: it assumes no crash happens during recovery itself. Test cases will always allow recovery to run to completion. (Real ARIES handles crashes during recovery using Compensation Log Records, which we do not implement here.)

6 Updates to Existing Layers

6.1 DiskSpaceManager

- Replace the `log_write` stub with a real method that the RecoveryManager uses to append log records to a dedicated log file.
- The log file is separate from data files and *never* enters the buffer pool.
- Expose `fsync` explicitly so the RecoveryManager can force the log to disk.

6.2 BufferManager

- Each frame must track the `pageLSN` of its page.
- Before writing a dirty page to disk (eviction or explicit flush), call `recovery.flush_log_up_to(page.pageLSN)`.
- Notify the RecoveryManager when a page becomes clean on disk so the Dirty Page Table can be updated.

6.3 FileIndexManager

- Before every page modification, call `recovery.log_update(...)` with the XID, page identifier, offset, before-image, and after-image.
- Set the page's `pageLSN` to the LSN returned by that call.

6.4 QueryProcessor

All transactions in Project 4 are explicit. Every operation must be wrapped in a `tx_begin` / `tx_op` / `tx_commit` block (see Section 7). The QueryProcessor maintains a mapping from transaction *name* (a label chosen by the input file, e.g. `T1`, `alice`) to internal XID. Multiple transactions may be open at the same time; their operations may interleave freely line by line. This is how test cases exercise concurrent-Xact recovery without requiring real threading.

7 New System Commands

These commands extend the Project 3 input language. They are mandatory and will appear in test cases. Commands fall into two groups: **transaction control** (which makes recovery testable) and **debugging / recovery** (which makes recovery observable).

Transaction control

Command	Effect
<code>tx_begin <name></code>	Open a new transaction labeled <code><name></code> . The label is chosen by the input file (e.g., <code>T1</code> , <code>alice</code>) and is local to the input language; internally it maps to an <code>XID</code> . Multiple transactions may be open at the same time.
<code>tx_op <name> <command></code>	Execute <code><command></code> (any Project 3 data operation) inside the open transaction <code><name></code> . The operation is logged under that transaction's <code>XID</code> . Does not commit.
<code>tx_commit <name></code>	Commit the open transaction <code><name></code> . Writes the commit record, fsyncs the log, then writes the end record. <code><name></code> is removed from the set of open transactions.

All data operations (`create record`, `delete record`, `search record`, `range_search`, `create type`) must appear as the body of a `tx_op` inside an open transaction. A data operation that appears outside any open transaction is invalid and must be logged as a failure in `log.csv`, with no effect on the database.

Example use:

```
tx_begin T1
tx_op T1 create record book Dune Herbert 1965 100
tx_begin T2
tx_op T2 create record book Foundation Asimov 1951 50
tx_commit T1
crash
```

After recovery on this input: `Dune` survives (`T1` committed before the crash); `Foundation` does not (`T2` was a loser).

Crash simulation

The `crash` command takes no arguments and simulates a power failure by calling `os._exit(1)` immediately. No log flushes run, no destructors run, no cleanup runs — the process is terminated mid-flight, exactly as if the machine had lost power. Any open `tx_begin` transactions at the time of the crash become losers and will be rolled back by the next recovery.

It must be `os._exit(1)`, not `sys.exit` and not an exception: those latter two run cleanup handlers, which defeats the purpose of the test.

8 Configuration File

```
{
  "page_size": 4096,
  "max_records_per_page": 10,
  "buffer_pool_size": 16,
  "replacement_policy": "LRU",
  "index_strategy": "bplus_tree",

  "checkpoint_interval": 50,
  "log_buffer_size": 8
}
```

Parameter	Values	Effect
<code>checkpoint_interval</code>	integer	Operations between fuzzy checkpoints.
<code>log_buffer_size</code>	integer	Log records held in RAM before flush.

9 Output Format and Ordering

Because grading diffs `output.txt` byte-for-byte against an expected file, all output must be deterministic and identical across implementations.

Record formatting.

- Each record appears on its own line.
- Fields are separated by single spaces in the order they were declared in `create type`.
- Integer fields are rendered in base 10 with no padding or sign for non-negatives; string fields are rendered as-is.
- `output.txt` ends with a single trailing newline.

search record. Prints the matching record on one line. If no record matches, prints nothing (no blank line).

range_search. Prints matching records in non-decreasing order of the field being searched. If two records share the same value in that field, ties are broken by primary key in ascending order. If no records match, prints nothing. This ordering rule applies regardless of the active index strategy: `heap_scan`, `hash_index` (which falls back to `heap_scan` for ranges), and `bplus_tree` must all produce the same output.

Multi-query inputs. Output for each query appears in the order the corresponding input line was processed. A `tx.op <name> range_search ...` produces its output between the surrounding queries' outputs and is never interleaved with another query's results.

10 Test Cases

A set of test cases will be provided. Each test case is a directory of the form:

```
case_<name>/
|-- config.json
|-- input_a.txt           # first input file (may or may not end in
    'crash')
|-- input_b.txt           # optional second input file (may end in
    'crash')
|-- ...                   # additional input files in lexicographic
    order
|-- verify.txt            # final input file: queries only
|-- expected_output.txt   # expected contents of output.txt after '
    verify.txt'
```

A test case consists of one or more input files followed by a single *verify* input file. The input files are run in lexicographic order on the same data directory. Each input file may end with a `crash` command or terminate normally; in either case the database state must remain consistent across the boundary, because each restart of `archive.py` runs recovery before processing the next file. The verify file contains only queries (no writes, no `crash`) and runs to clean completion.

To run a test case manually:

```
# Start from an empty data directory.
python3 archive.py case_foo/config.json case_foo/input_a.txt
python3 archive.py case_foo/config.json case_foo/input_b.txt
...
python3 archive.py case_foo/config.json case_foo/verify.txt
# Compare output.txt to case_foo/expected_output.txt
```

Each `archive.py` invocation may exit with a non-zero status if the input file ends in `crash`; this is expected. Only the final verify run must complete normally. The output file after each `archive.py` run can be overwritten after each input, the important part is that the database is consistent. After the verify run, `output.txt` must match `expected_output.txt` byte-for-byte (see Section 8 for the format and ordering rules).

Test cases will cover: durability of committed transactions across a crash, rollback of uncommitted transactions, recovery across a fuzzy checkpoint, recovery across multiple consecutive crash-restart cycles, and interleaved transactions where some commit before the crash and some do not. The same contract applies to every case: produce the expected `output.txt` after the verify run.

11 Output File Locations

All output files are written to the same directory as `archive.py`:

- `output.txt` — query results.
- `log.csv` — persistent operation log (from Project 3).
- `wal.log` — the WAL log file (new).
- All data files (relation files, index files, system catalog, master record).

12 Technical Constraints

- Standard library only. No third-party packages.
- All input files use ASCII-compatible encoding.
- LSNs must persist across restarts: do not reset to zero.
- The log file must not be cached in the buffer pool.
- `commit` must use `os.fsync`, not a plain `flush`.
- `crash` must use `os._exit(1)`, not `sys.exit` and not an exception.

13 Persistence

All data files, the system catalog, index structures, the WAL log, the master record, and `log.csv` must persist across runs. On restart, the engine must recover its state from disk via the three-phase recovery algorithm.

14 Submission

Submit one zip file with the code in it. We will run the hidden test cases against your `archive.py` file.

15 Individual Contribution Report Guidelines

Each student must prepare an Individual Contribution Report and include it in the group ZIP file. The report must be in PDF format, named: `Student_ID_Contribution.pdf`.

What to Include:

- Personal information: Name, Student ID, Group Number.
- Tasks and contributions: which parts of the Recovery Manager you implemented (log records, Analysis, Redo, Undo, checkpointing, documentation, debugging).
- Collaboration and challenges.
- Self-assessment: reflection on your role, skills learned, areas for improvement.
- Use of AI tools: which tools, what you asked, what you changed.

Formatting: 1 page (max 2), Times New Roman 12pt, 1.5 spacing, PDF.

16 Optional Project Policy

This project is **optional**.

Participation. You may choose whether or not to participate. You may work on this project individually or with a partner. If your group partner does not want to participate, you are allowed to complete the project alone.

Effect on letter grade. The bonus points from this project will not be used when determining grade cutoffs (letter-grade thresholds). Letter grades will be decided based on your main course performance, and the bonus will be applied on top. Choosing not to participate will not affect your letter grade. Completing the project may improve your letter grade, although this is not guaranteed.

Bonus calculation. Successfully completing this project can earn you a **10% bonus** toward your overall course grade, *provided that your project grade is at least 80 out of 100*. For example, if your project grade is 90, then $90 \times 10\% = 9$ points will be added to your overall course grade. If this increases your overall grade enough to place you in a higher letter-grade range, your letter grade will be updated accordingly.

Below the threshold. If your project grade is below 80, it will *not* be included in the calculation of your overall course grade and will not affect your letter grade.

Deadline. June 9, 2026, 17:00 sharp. No late submissions will be accepted. No extensions will be given under any circumstances.

17 AI Usage

AI tools are permitted as learning aids. Include `ai_usage.md` describing what you used, what you asked, and what you changed. Honest disclosure is not penalized. You must be able to explain every part of your code, including the Recovery Manager.